



Politechnika Krakowska im. Tadeusza Kościuszki

Wydział Informatyki i Telekomunikacji

PRACA MAGISTERSKA

Zastosowanie fine-tuning'u dużych modeli językowych do wykrywania niechcianej poczty elektronicznej

Title consistent with the topic/field of the thesis

Autor:	Wojciech Drózdź
Kierunek:	Informatyka
Opiekun pracy:	dr inż. Dariusz Żelasko

Kraków, 2025

Tutaj możesz umieścić treść podziękowań. Tutaj możesz umieścić treść podziękowań. Tutaj możesz umieścić treść podziękowań. Tutaj możesz umieścić treść podziękowań.

Streszczenie

Streszczenie po polsku ...

Abstract

Abstract in English . . .

Spis treści

Spis rysunków	xiii
Spis tabel	xv
Lista kodów źródłowych	xix
1. Spam i co dalej?	1
1.1. Problem niechcianej poczty elektronicznej (spamu)	1
1.1.1. Definicja i rodzaje spamu	1
1.1.2. Skutki spamu i znaczenie jego filtrowania	1
1.2. Duże modele językowe (LLM) w przetwarzaniu języka naturalnego	1
1.2.1. Podstawowe koncepcje LLM	1
1.2.2. Fine-tuning jako metoda adaptacji modeli	1
1.3. Cel i zakres pracy	1
1.3.1. Główny cel badawczy	1
1.3.2. Pytania badawcze	1
1.3.3. Zakres i ograniczenia pracy	1
1.4. Struktura pracy	1
2. Przegląd istniejących metod wykrywania spamu	3
2.1. Tradycyjne metody filtrowania	4
2.1.1. Filtry oparte na regułach	4
2.1.2. Filtry statystyczne (np. Bayesowskie)	4
2.1.3. Listy czarne i białe (Black/White Lists)	4
2.2. Metody oparte na klasycznym uczeniu maszynowym	4
2.2.1. Modele wektorów nośnych (SVM)	4
2.2.2. Drzewa decyzyjne i lasy losowe	4
2.2.3. Inne algorytmy klasyfikacji	4
2.3. Metody oparte na głębokim uczeniu	4
2.3.1. Sieci neuronowe konwolucyjne (CNN) w analizie tekstu	4
2.3.2. Sieci neuronowe rekurencyjne (RNN, LSTM, GRU)	4
2.3.3. Modele oparte na architekturze Transformer (przed erą LLM na dużą skalę)	4

2.4.	Wykorzystanie dużych modeli językowych (LLM) do klasyfikacji tekstu . . .	4
2.4.1.	Podejścia zero-shot i few-shot	4
2.4.2.	Fine-tuning LLM do zadań specyficznych	4
2.4.3.	Dotychczasowe badania nad LLM w kontekście spamu	4
2.5.	Podsumowanie i identyfikacja luki badawczej	4
3.	Część badawcza	5
3.1.	Wybór otwartoźródłowego modelu językowego	6
3.1.1.	Przegląd dostępnych modeli (np. Llama, Mistral, Falcon, itp.)	6
3.1.2.	Kryteria wyboru modelu (wydajność, rozmiar, licencja, możliwości fine-tuning'u)	6
3.1.3.	Uzasadnienie wyboru konkretnego modelu	6
3.2.	Metodologia badawcza	6
3.2.1.	Opis wykorzystanych zbiorów danych	6
3.2.2.	Przygotowanie danych (Preprocessing)	6
3.2.3.	Proces fine-tuning'u modelu	6
3.2.4.	Metryki oceny skuteczności	6
3.3.	Implementacja i przeprowadzenie eksperymentów	6
3.3.1.	Środowisko eksperymentalne	6
3.3.2.	Przebieg procesu fine-tuning'u	7
3.4.	Analiza skuteczności dostrojonego modelu	7
3.4.1.	Wyniki na zbiorze testowym	7
3.4.2.	Porównanie z wynikami istniejących metod (z Rozdziału 2)	7
3.4.3.	Analiza błędów klasyfikacji (False Positives, False Negatives)	7
3.4.4.	Ocena elastyczności modelu i odporności na nowe typy spamu	7
3.5.	Analiza możliwości wdrożenia produkcyjnego	7
3.5.1.	Wymagania sprzętowe i czas wnioskowania (inference time)	7
3.5.2.	Koszty obliczeniowe (fine-tuning i wnioskowanie)	7
3.5.3.	Skalowalność rozwiązania	7
3.5.4.	Potencjalne wyzwania i ograniczenia wdrożeniowe	7
3.5.5.	Praktyczność zastosowania w porównaniu do tradycyjnych filtrów . .	7
4.	Podsumowanie i wnioski	9
4.1.	Synteza przeprowadzonych badań i uzyskanych wyników	9
4.2.	Odpowiedzi na pytania badawcze	9
4.3.	Wnioski dotyczące skuteczności i praktyczności fine-tuning'u LLM	9
4.4.	Ograniczenia pracy	9
4.5.	Kierunki dalszych badań	9

A. Typowe elementy składowe pracy dyplomowej z informatyki	11
A.1. Tabele	11
A.2. Rysunki	13
A.2.1. Wewnętrzne	13
A.2.2. Zewnętrzne	14
A.3. Kody źródłowe	14
A.4. Algorytmy	16
A.5. Wzory	16
A.5.1. Przykłady	17
A.6. Twierdzenia i podobne struktury	17
Uwagi Autora	19
Bibliografia	21

Zawartość spisu treści — tytuły rozdziałów oraz ich liczba zależą od tematyki pracy — należy ustalić z opiekunem pracy.

Spis rysunków

A.1. Prostý rysunek <i>TikZ</i>	13
A.2. Bardziej złożony rysunek <i>TikZ</i>	13
A.3. Logo Wydziału Informatyki.	14

Spis tabel

A.1. Pomiary zużycia energii elektrycznej.	11
A.2. Tabela, która zawiera dużą liczbę wierszy.	11
A.3. Tabela zawierająca długi tekst.	12

Lista algorytmów

1. Disjoint decomposition. 16

Lista kodów źródłowych

A.1. Przykładowy kod źródłowy sformatowany za pomocą pakietu 'listings' . . .	15
A.1. Przykładowy listing sformatowany za pomocą pakietu 'minted'	15

1. Spam i co dalej?

1.1. Problem niechcianej poczty elektronicznej (spamu)

1.1.1. Definicja i rodzaje spamu

1.1.2. Skutki spamu i znaczenie jego filtrowania

1.2. Duże modele językowe (LLM) w przetwarzaniu języka naturalnego

1.2.1. Podstawowe koncepcje LLM

1.2.2. Fine-tuning jako metoda adaptacji modeli

1.3. Cel i zakres pracy

1.3.1. Główny cel badawczy

1.3.2. Pytania badawcze

1.3.3. Zakres i ograniczenia pracy

1.4. Struktura pracy

2. Przegląd istniejących metod wykrywania spamu

2.1. Tradycyjne metody filtrowania

2.1.1. Filtry oparte na regułach

2.1.2. Filtry statystyczne (np. Bayesowskie)

2.1.3. Listy czarne i białe (Black/White Lists)

2.2. Metody oparte na klasycznym uczeniu maszynowym

2.2.1. Modele wektorów nośnych (SVM)

2.2.2. Drzewa decyzyjne i lasy losowe

2.2.3. Inne algorytmy klasyfikacji

2.3. Metody oparte na głębokim uczeniu

2.3.1. Sieci neuronowe konwolucyjne (CNN) w analizie tekstu

2.3.2. Sieci neuronowe rekurencyjne (RNN, LSTM, GRU)

2.3.3. Modele oparte na architekturze Transformer (przed erą LLM na dużą skalę)

2.4. Wykorzystanie dużych modeli językowych (LLM) do klasyfikacji tekstu

2.4.1. Podejścia zero-shot i few-shot

2.4.2. Fine-tuning LLM do zadań specyficznych

2.4.3. Dotychczasowe badania nad LLM w kontekście spamu

2.5. Podsumowanie i identyfikacja luki badawczej

3. Część badawcza

3.1. Wybór otwartoźródłowego modelu językowego

3.1.1. Przegląd dostępnych modeli (np. Llama, Mistral, Falcon, itp.)

3.1.2. Kryteria wyboru modelu (wydajność, rozmiar, licencja, możliwości fine-tuning'u)

3.1.3. Uzasadnienie wyboru konkretnego modelu

3.2. Metodologia badawcza

3.2.1. Opis wykorzystanych zbiorów danych

Zbiór treningowy

Zbiór walidacyjny

Zbiór testowy

3.2.2. Przygotowanie danych (Preprocessing)

3.2.3. Proces fine-tuning'u modelu

Architektura zadania klasyfikacyjnego

Parametry i hiperparametry uczenia

Wykorzystane narzędzia i biblioteki

3.2.4. Metryki oceny skuteczności

Dokładność (Accuracy)

Precyzja (Precision)

Pełność (Recall)

Miara F1 (F1-Score)

Inne istotne metryki (np. AUC)

3.3. Implementacja i przeprowadzenie eksperymentów

3.3.1. Środowisko eksperymentalne

3.3.2. Przebieg procesu fine-tuning'u

KUUURWA

3.4. Analiza skuteczności dostrojonego modelu

3.4.1. Wyniki na zbiorze testowym

3.4.2. Porównanie z wynikami istniejących metod (z Rozdziału 2)

3.4.3. Analiza błędów klasyfikacji (False Positives, False Negatives)

3.4.4. Ocena elastyczności modelu i odporności na nowe typy spamu

3.5. Analiza możliwości wdrożenia produkcyjnego

3.5.1. Wymagania sprzętowe i czas wnioskowania (inference time)

3.5.2. Koszty obliczeniowe (fine-tuning i wnioskowanie)

3.5.3. Skalowalność rozwiązania

3.5.4. Potencjalne wyzwania i ograniczenia wdrożeniowe

3.5.5. Praktyczność zastosowania w porównaniu do tradycyjnych filtrów

4. Podsumowanie i wnioski

- 4.1. Synteza przeprowadzonych badań i uzyskanych wyników**
- 4.2. Odpowiedzi na pytania badawcze**
- 4.3. Wnioski dotyczące skuteczności i praktyczności fine-tuning'u LLM**
- 4.4. Ograniczenia pracy**
- 4.5. Kierunki dalszych badań**

Dodatek A.

Typowe elementy składowe pracy dyplomowej z informatyki

A.1. Tabele

Uwaga A.1.

- Każda tabela powinna być opisana w treści pracy.
- Podpis ma być przed tabelą.

W tabeli [A.1](#) przedstawiono wyniki pomiarów.

Tabela A.1.: Pomiary zużycia energii elektrycznej.

L.p.	Wartość
1	12345,6789
	45,89
2	45,678901

Jeżeli tabela zawiera dużą liczbę wierszy i może nie zmieścić się na stronie — patrz tabela [A.2](#) — skorzystaj z pakietu *longtable* [\[1\]](#).

Tabela A.2.: Tabela, która zawiera dużą liczbę wierszy.

	1	2	3	4	5	6	7	8	
Student 1									

	1	2	3	4	5	6	7	8	
Student 2									
Student 3									
Student 4									
Student 5									
Student 6									
Student 7									
Student 8									
Student 9									

Tabele, w których występuje długi tekst, a co za tym idzie może się on nie zmieścić — musi zostać zawinięty, z pomocą przychodzi środowisko 'tabularx' [2] — patrz tabela A.3.

Tabela A.3.: Tabela zawierająca długi tekst.

Wpis wielokolumnowy!		TRZY	CZTERY
jeden	Szerokość tej kolumny zależy od szerokości tabeli.	trzy	Kolumna czwarta będzie zachowywać się w taki sam sposób jak druga kolumna o tej samej szerokości.

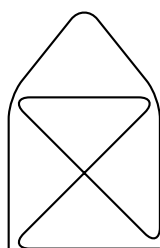
A.2. Rysunki

Uwaga A.2.

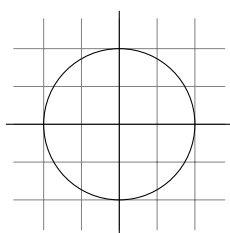
- Rysunki powinny być przerysowane samodzielnie albo używane tylko te, których twórcy zezwolili na ich rozpowszechnianie oraz kopiowanie, czyli np. rysunki objęte licencją Creative Commons.
- Każdy rysunek powinien być opisany w treści pracy.

A.2.1. Wewnętrzne

Klasa *agh-wi*, automatycznie, dołącza pakiet *TikZ* [3] — dostarcza on komend pozwalających na tworzenie grafik. Przykładowe grafiki pokazano na rysunku A.1 oraz A.2.



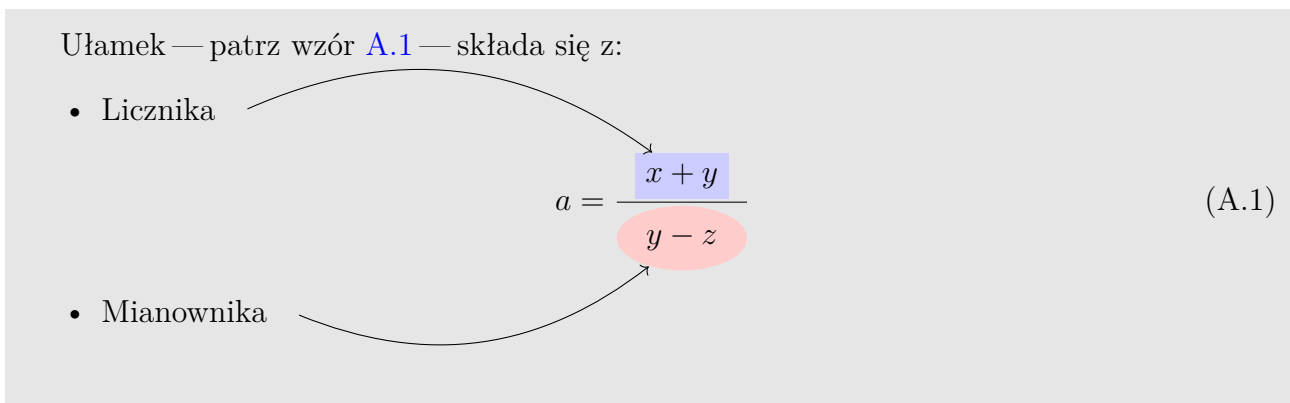
Rysunek A.1.: Prosty rysunek *TikZ*.



Rysunek A.2.: Bardziej złożony rysunek *TikZ*.

Oprócz rysunków eksponowanych możliwe jest tworzenie grafik będących  częścią zdania.

TikZ pozwala również na kreślenie po powierzchni strony, np. możemy narysować strzałki pomiędzy elementami strony.



A.2.2. Zewnętrzne

Oczywiście możliwe jest również dołączanie rysunków zewnętrznych — pakiet *graphics* [4] pozwala na wstawianie grafik zapisanych w plikach: '.png', '.jpg' oraz '.pdf'. Rysunek A.3 wstawiono przy użyciu tego pakietu.



Rysunek A.3.: Logo Wydziału Informatyki.

A.3. Kody źródłowe

Najpopularniejszymi pakietami, które umożliwiają składanie kodów źródłowych programów, są:

listings [5] — kod źródłowy jest formatowany bezpośrednio przez $\text{\LaTeX}$ -a — nie jest używany żaden, zewnętrzny, formater kodu.

Kod źródłowy A.1: Przykładowy kod źródłowy sformatowany za pomocą pakietu 'listings'.

```

1  /* Pierwszy program w C++ */
2
3  #include <iostream>
4
5  int main() {
6      std::cout << "Hello World!";
7      return 0;
8  }
```

minted [6] — formatuje kod źródłowy przy użyciu biblioteki języka Python o nazwie *Pygments* [7].

Kod źródłowy A.1.: Przykładowy listing sformatowany za pomocą pakietu 'minted'.

```

1  /* Pierwszy program w C++ */
2
3  #include <iostream>
4
5  int main() {
6      std::cout << "Hello World!";
7      return 0;
8  }
```

Uwaga A.3.

- Podpis ma być przed kodem źródłowym.
- **Proszę używać tylko jednego z tych pakietów**; w przeciwnym razie otrzymasz taki efekt, jak w przykładowej pracy — obydwie listingi mają ten sam numer.

Kod źródłowy w C++ sformatowany przy użyciu pakietu *listings*, pokazano na listingu [A.1](#); sformatowany przy użyciu pakietu *minted*, pokazano na listingu [A.1](#).

A.4. Algorytmy

Pakiet *algorithm2e* [8] to jeden z kilku, które pozwalają zapisywać algorytmy w formie pseudokodu — patrz algorytm 1.

Uwaga A.4. Podpis ma być przed algorytmem.

Algorytm 1: Disjoint decomposition.

```
input : A bitmap  $Im$  of size  $w \times l$ 
output: A partition of the bitmap
1 special treatment of the first line;
2 for  $i \leftarrow 2$  to  $l$  do
3   special treatment of the first element of line  $i$ ;
4   for  $j \leftarrow 2$  to  $w$  do
5      $\text{left} \leftarrow \text{FindCompress}(Im[i, j - 1]);$ 
6      $\text{up} \leftarrow \text{FindCompress}(Im[i - 1, ]);$ 
7      $\text{this} \leftarrow \text{FindCompress}(Im[i, j]);$ 
8     if  $\text{left}$  compatible with this then //  $0(\text{left}, \text{this}) == 1$ 
9       if  $\text{left} < \text{this}$  then  $\text{Union}(\text{left}, \text{this});$ 
10      else  $\text{Union}(\text{this}, \text{left});$ 
11    end
12    if  $\text{up}$  compatible with this then //  $0(\text{up}, \text{this}) == 1$ 
13      if  $\text{up} < \text{this}$  then  $\text{Union}(\text{up}, \text{this});$ 
14      // this is put under up to keep tree as flat as possible
15      else  $\text{Union}(\text{this}, \text{up});$ 
16      // this linked to up
17    end
18  end
19 foreach element  $e$  of the line  $i$  do  $\text{FindCompress}(p);$ 
20 end
```

A.5. Wzory

L^AT_EX bardzo dobrze sprawdza się w przypadku prac dyplomowych zawierających wzory matematyczne¹.

¹W przypadku złożonych wzorów warto zastosować pakiet *amsmath* [9].

A.5.1. Przykłady

Wzór $E = mc^2$ jest częścią zdania.

$$\left| \sum_{i=1}^n a_i b_i \right| \leq \left(\sum_{i=1}^n a_i^2 \right)^{1/2} \left(\sum_{i=1}^n b_i^2 \right)^{1/2} \quad (\text{A.2})$$

Wartości zmiennej opisano wzorem A.3.

$$x = \begin{cases} y & \text{dla } y > 0 \\ \frac{z}{y} & \text{dla } y \leq 0 \end{cases} \quad (\text{A.3})$$

Wzór A.4 to wzór wielowierszowy.

$$\begin{aligned} 2x^2 + 3(x-1)(x-2) &= 2x^2 + 3(x^2 - 3x + 2) \\ &= 2x^2 + 3x^2 - 9x + 6 \\ &= 5x^2 - 9x + 6 \end{aligned} \quad (\text{A.4})$$

Uwaga A.5. Należy używać tylko dwóch rodzajów wzorów:

1. „W linii”.
2. Eksponowane, numerowane.

A.6. Twierdzenia i podobne struktury

Twierdzenie nr 1 opublikował, w roku 1691, francuski matematyk Michel Rolle.

Twierdzenie 1 (Rolle’a) *Jeśli dana funkcja $f: \mathbb{R} \rightarrow \mathbb{R}$ jest:*

1. ciągła w przedziale $[a, b]$
 2. jest różniczkowalna w przedziale (a, b)
 3. na końcach przedziału $[a, b]$ przyjmuje równe wartości: $f(a) = f(b)$,
- to w przedziale (a, b) istnieje co najmniej jeden punkt c taki, że $f'(c) = 0$.

Teraz coś z informatyki ...

Definicja 1 *Bit to najmniejsza jednostka informacji w komputerze.*

Definicja 2 *Bajtem nazywamy ciąg ośmiu bitów.*

Uwagi Autora

- Aktualna wersja klasy jest dostępna pod adresem <https://github.com/polaksta/LaTeX/tree/master/agh-wi>¹.
- Skoro Twoja praca dyplomowa powstała w L^AT_EXu, to zachęcam Cię również do przygotowania prezentacji (na obronę pracy magisterskiej) w tym języku. Witaj. Najpopularniejszą klasą do tworzenia tego typu dokumentów jest *beamer* [10].
- Pod adresem <https://github.com/polaksta/LaTeX/tree/master/beamerthemeAGH>² możesz znaleźć, stworzony przeze mnie, nasz uczelniany szablon dla prezentacji L^AT_EX Beamer.
- Jeżeli pewne elementy mają być wyróżniane w **jednakowy** **sposób**, to proponuję nie używać bezpośredniego stylowania, tzn.

1 `\colorbox{red!50}{jednakowy} \colorbox{red!50}{sposób}`

ale zdefiniować własną komendę stylującą, np. `\alert`,

1 `\newcommand{\alert}[1]{\colorbox{red!50}{#1}}`

a następnie użyć jej w dokumencie.

1 `\alert{jednakowy} \alert{sposób}`

Dzięki temu, jeżeli będziesz chciał / chciała zmienić sposób stylowania tych elementów, np. niebieskie tło zamiast czerwonego, to wystarczy zmodyfikować, tylko, definicję komendy, zamiast zastępować, w tekście pracy dyplomowej, wybrane (niekoniecznie wszystkie!) wystąpienia tekstu `red`, tekstem `blue`.

Stanisław Polak

¹W przypadku Overleaf-a jest ona pod adresem <https://www.overleaf.com/read/fnvcvqjyrbyw#5ac622>

²W przypadku Overleaf-a jest on pod adresem <https://www.overleaf.com/read/fkjdtbnbrfhj#9c6184>

Bibliografia

- [1] *The longtable package*. URL: <http://mirrors.ctan.org/macros/latex/required/tools/longtable.pdf>.
- [2] *The tabularx package*. URL: <http://mirrors.ctan.org/macros/latex/required/tools/tabularx.pdf>.
- [3] *The TikZ and PGF Packages*. URL: <http://mirrors.ctan.org/graphics/pgf/base/doc/pgfmanual.pdf>.
- [4] *Packages in the ‘graphics’ bundle*. URL: <http://mirrors.ctan.org/macros/latex/required/graphics/grfguide.pdf>.
- [5] *The Listings Package*. URL: <http://mirrors.ctan.org/macros/latex/contrib/listings/listings.pdf>.
- [6] *The minted package: Highlighted source code in L^AT_EX*. URL: <http://mirrors.ctan.org/macros/latex/contrib/minted/minted.pdf>.
- [7] *Strona WWW biblioteki „Pygments”*. URL: <https://pygments.org/>.
- [8] *algorithm2e.sty — package for algorithms*. URL: <http://mirrors.ctan.org/macros/latex/contrib/algorithm2e/doc/algorithm2e.pdf>.
- [9] *User’s Guide for the amsmath Package*. URL: <http://mirrors.ctan.org/macros/latex/required/amsmath/amslldoc.pdf>.
- [10] *The beamer class*. URL: <http://mirrors.ctan.org/macros/latex/contrib/beamer/doc/beameruserguide.pdf>.